

```

# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

/kaggle/input/nlp-getting-started/sample_submission.csv
/kaggle/input/nlp-getting-started/train.csv
/kaggle/input/nlp-getting-started/test.csv

#Extracting training data from csv to a pandas dataframe
df_train= pd.read_csv("/kaggle/input/nlp-getting-started/train.csv")
df_train

```

	id	keyword	location	\
0	1	NaN	NaN	
1	4	NaN	NaN	
2	5	NaN	NaN	
3	6	NaN	NaN	
4	7	NaN	NaN	
...	...	...	...	
7608	10869	NaN	NaN	
7609	10870	NaN	NaN	
7610	10871	NaN	NaN	
7611	10872	NaN	NaN	
7612	10873	NaN	NaN	

		text	target
0	Our Deeds are the Reason of this #earthquake M...		1
1	Forest fire near La Ronge Sask. Canada		1
2	All residents asked to 'shelter in place' are ...		1

```

3      13,000 people receive #wildfires evacuation or...      1
4      Just got sent this photo from Ruby #Alaska as ...      1
...
7608   Two giant cranes holding a bridge collapse int...      1
7609   @aria_ahrary @TheTawniest The out of control w...      1
7610   M1.94 [01:04 UTC]?5km S of Volcano Hawaii. htt...      1
7611   Police investigating after an e-bike collided ...      1
7612   The Latest: More Homes Razed by Northern Calif...      1

```

```
[7613 rows x 5 columns]
```

#Extracting testing data from csv to a pandas dataframe

```

df_test= pd.read_csv("/kaggle/input/nlp-getting-started/test.csv")
df_test.head()

```

```

      id keyword location
text
0    0      NaN      NaN      Just happened a terrible car
crash
1    2      NaN      NaN  Heard about #earthquake is different cities,
s...
2    3      NaN      NaN  there is a forest fire at spot pond, geese
are...
3    9      NaN      NaN      Apocalypse lighting. #Spokane
#wildfires
4   11      NaN      NaN      Typhoon Soudelor kills 28 in China and
Taiwan

```

Taking a quick look at our data to understand what a non-disaster and a disaster tweet look like

```

df_train[df_train["target"] == 0]["text"].values[1]
'I love fruits'

df_train[df_train["target"] == 1]["text"].values[1]
'Forest fire near La Ronge Sask. Canada'

```

Building vectors

For machine to understand whether a tweet indicates disaster or not it first needs to understand the impact of the type and the arrangement of the words in a tweet on the "target" values. For this, we first need to convert each word in a tweet in the vector format that a machine can easily understand.

```

import tensorflow as tf

df_train_subset = df_train.iloc[:5800]
df_valid_subset = df_train.iloc[5800:]

```

```

all_dataset =
tf.data.Dataset.from_tensor_slices((df_train["text"].tolist(),
df_train["target"].tolist()))
train_dataset =
tf.data.Dataset.from_tensor_slices((df_train_subset["text"].tolist(),
df_train_subset["target"].tolist()))
validation_dataset =
tf.data.Dataset.from_tensor_slices((df_valid_subset["text"].tolist(),
df_valid_subset["target"].tolist()))
test_dataset =
tf.data.Dataset.from_tensor_slices((df_test["text"].tolist()))

#Defining useful Global Variables

VOCAB_SIZE = 1000
EMBEDDING_DIM = 16
MAX_LENGTH = 120

def standardize_func(sentence):
    """Removes stopwords and punctuation from a sentence.

Args:
    sentence: A TensorFlow string tensor.

Returns:
    A TensorFlow string tensor with stopwords and punctuation removed.
    """

    # List of stopwords
    stopwords = ["a", "about", "above", "after", "again", "against",
    "all", "am"]
    # Create a regular expression pattern to match stopwords and
punctuation
    stopword_pattern = r"\b(" + "|".join(stopwords) + r")\b|[\^\w\s]"

    # Remove stopwords and punctuation in a single step
    sentence = tf.strings.regex_replace(sentence, stopword_pattern, "")

    # Convert the sentence to lowercase
    sentence = tf.strings.lower(sentence)

    return sentence

def fit_vectorizer(train_sentences, standardize_func):

    # Instantiate the TextVectorization class, passing in the correct
values for
    vectorizer = tf.keras.layers.TextVectorization(
        standardize=standardize_func,
        max_tokens=VOCAB_SIZE,
        output_sequence_length=MAX_LENGTH
    )

```

```

    # Adapt the vectorizer to the training sentences
    vectorizer.adapt(train_sentences)

    return vectorizer

# Create the vectorizer
text_only_dataset = all_dataset.map(lambda text, label: text)
concatenated_dataset = text_only_dataset.concatenate(test_dataset)
vectorizer = fit_vectorizer(concatenated_dataset, standardize_func)

#test_vectorizer = fit_vectorizer(test_dataset, standardize_func)

def fit_label_encoder(train_labels, validation_labels):

    # Combine the train and validation labels
    labels = tf.data.Dataset.concatenate(train_labels,
validation_labels)

    return labels

train_labels_only = train_dataset.map(lambda text, label: label)
validation_labels_only = validation_dataset.map(lambda text, label:
label)
label_encoder =
fit_label_encoder(train_labels_only, validation_labels_only)

# GRADED FUNCTION: preprocess_dataset
def preprocess_dataset(dataset, text_vectorizer, label_encoder):

    def preprocess_text_label_pair(text, label):
        """Preprocesses a single text-label pair."""
        preprocessed_text = text_vectorizer(text)
        preprocessed_label = label
        return preprocessed_text, preprocessed_label
    # Convert the Datasetmap(lambda text, label: label) sentences to
sequences,
    dataset = dataset.map(preprocess_text_label_pair)
    dataset = dataset.batch(32) # Set a batchsize of 32

    return dataset

test_dataset = test_dataset.map(vectorizer)
test_proc_dataset = test_dataset.batch(32)

train_proc_dataset = preprocess_dataset(train_dataset, vectorizer,
label_encoder)
validation_proc_dataset = preprocess_dataset(validation_dataset,
vectorizer, label_encoder)

```

Model Creation

```

def create_model():

    # Define your model
    model = tf.keras.Sequential([
        tf.keras.Input(shape=(MAX_LENGTH,)),
        tf.keras.layers.Embedding(VOCAB_SIZE, EMBEDDING_DIM),
        tf.keras.layers.Dropout(0.50),
        tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(96)),
        tf.keras.layers.Dense(24, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])
    # Compile model. Set an appropriate loss, optimizer and metrics
    model.compile(
        loss='binary_crossentropy',
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.003),
        metrics=['accuracy']
    )

    return model

model = create_model()

history = model.fit(train_proc_dataset, epochs=6,
                    validation_data=validation_proc_dataset)

Epoch 1/6
182/182 [=====] - 24s 107ms/step - loss:
0.6830 - accuracy: 0.5602 - val_loss: 0.6781 - val_accuracy: 0.5560
Epoch 2/6
182/182 [=====] - 18s 101ms/step - loss:
0.6462 - accuracy: 0.6478 - val_loss: 0.5164 - val_accuracy: 0.7485
Epoch 3/6
182/182 [=====] - 18s 101ms/step - loss:
0.4963 - accuracy: 0.7721 - val_loss: 0.4856 - val_accuracy: 0.7656
Epoch 4/6
182/182 [=====] - 18s 99ms/step - loss:
0.4440 - accuracy: 0.8005 - val_loss: 0.4875 - val_accuracy: 0.7750
Epoch 5/6
182/182 [=====] - 19s 106ms/step - loss:
0.4223 - accuracy: 0.8129 - val_loss: 0.4974 - val_accuracy: 0.7634
Epoch 6/6
182/182 [=====] - 19s 104ms/step - loss:
0.4086 - accuracy: 0.8224 - val_loss: 0.5164 - val_accuracy: 0.7617

sample_submission =
pd.read_csv("/kaggle/input/nlp-getting-started/sample_submission.csv")

sample_submission["target"] = model.predict(test_proc_dataset)

102/102 [=====] - 4s 32ms/step

```

```
sample_submission.head()
```

	id	target
0	0	0.815142
1	2	0.406720
2	3	0.963271
3	9	0.115209
4	11	0.998731

```
sample_submission['target'] = sample_submission['target'].apply(lambda  
x: 1 if x >= 0.5 else 0)
```

```
sample_submission.head()
```

	id	target
0	0	1
1	2	0
2	3	1
3	9	0
4	11	1

```
sample_submission.to_csv("submission.csv", index=False)
```